

Subject :- Cybersecurity
Practical 3

AIM:- Implement algorithms to generate and verify message authentication codes (MACs) for ensuring data integrity and authenticity.

CODE:-

```
import hmac
import hashlib

def generate_mac(key, message):
    return hmac.new(
        key.encode(),
        message.encode(),
        hashlib.sha256
    ).hexdigest()

def verify_mac(key, message, received_mac):
    computed_mac = generate_mac(key, message)
    return hmac.compare_digest(computed_mac, received_mac)

key = input("Enter the secret key: ")
message = input("Enter the message: ")

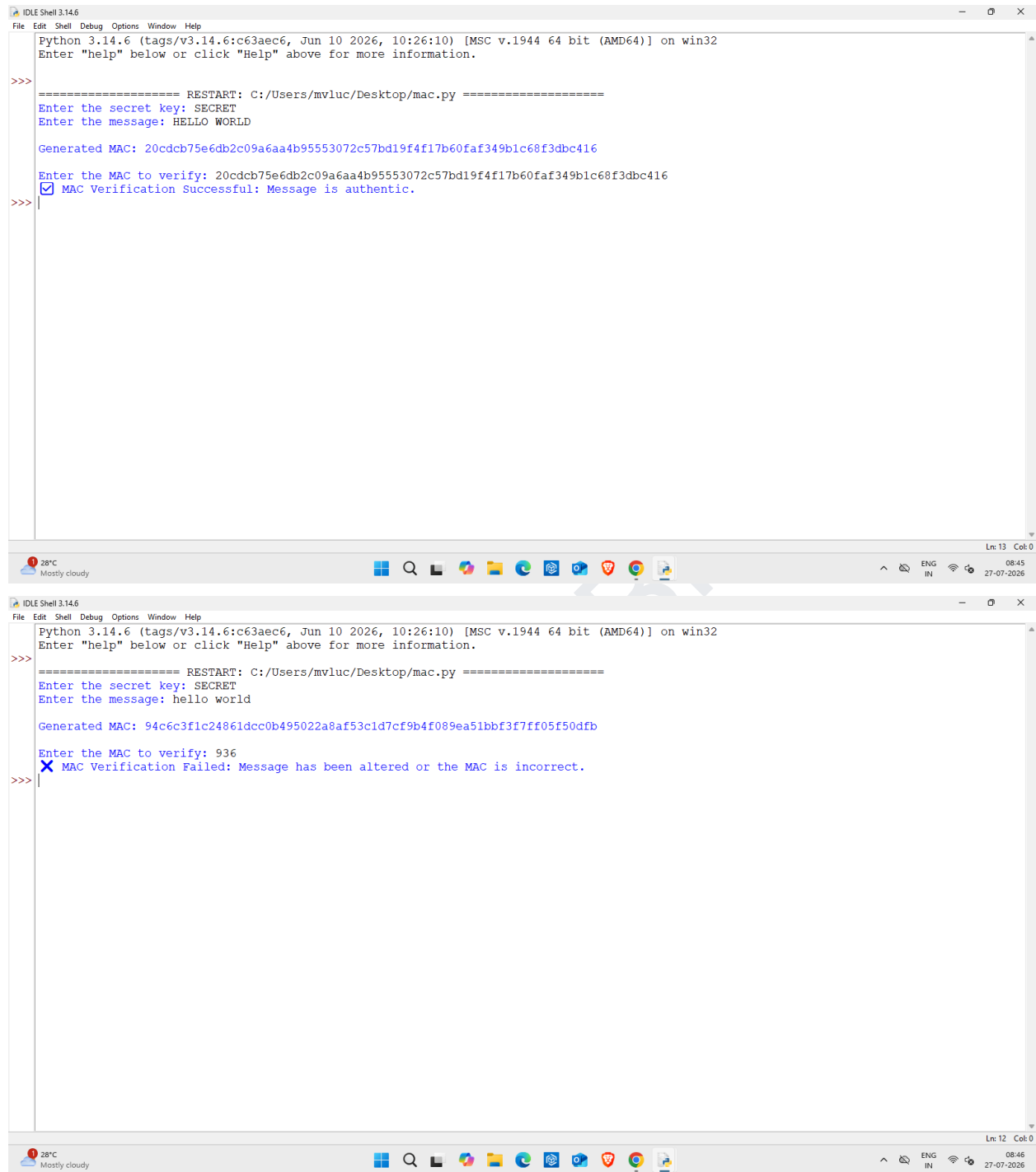
mac = generate_mac(key, message)
print("\nGenerated MAC:", mac)

received_mac = input("\nEnter the MAC to verify: ")

if verify_mac(key, message, received_mac):
    print("✅ MAC Verification Successful: Message is authentic.")
else:
    print("❌ MAC Verification Failed: Message has been altered or the MAC is incorrect.")
```

OUTPUT:-

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce



The image displays two screenshots of a Python 3.14.6 IDLE Shell window, illustrating a MAC (Message Authentication Code) generation and verification process.

Top Screenshot: The shell shows the execution of a script named `mac.py`. The user enters the secret key `SECRET` and the message `HELLO WORLD`. The script generates a MAC: `20cdb75e6db2c09a6aa4b95553072c57bd19f4f17b60faf349b1c68f3dbc416`. The user then enters this MAC for verification, and the script outputs: `MAC Verification Successful: Message is authentic.`

Bottom Screenshot: The shell shows the same script being executed again. The user enters the secret key `SECRET` and the message `hello world`. The script generates a MAC: `94c6c3f1c24861dcc0b495022a8af53c1d7cf9b4f089ea51bbf3f7ff05f50dfb`. The user then enters `936` for verification, and the script outputs: `MAC Verification Failed: Message has been altered or the MAC is incorrect.`

GUI CODE:-

```
import tkinter as tk
from tkinter import messagebox
import hmac
import hashlib
```

```
def generate_mac(key, message):
    return hmac.new(
        key.encode(),
        message.encode(),
        hashlib.sha256
    ).hexdigest()
```

```
def generate():
    key = key_entry.get()
    message = message_text.get("1.0", tk.END).strip()

    if not key or not message:
        messagebox.showerror("Error", "Please enter both Key and Message.")
        return

    mac = generate_mac(key, message)

    mac_entry.config(state="normal")
    mac_entry.delete(0, tk.END)
    mac_entry.insert(0, mac)
    mac_entry.config(state="readonly")
```

```
def verify():
    key = key_entry.get()
    message = message_text.get("1.0", tk.END).strip()
    entered_mac = verify_entry.get().strip()

    if not key or not message or not entered_mac:
        messagebox.showerror("Error", "Please fill all fields.")
        return

    computed_mac = generate_mac(key, message)

    if hmac.compare_digest(computed_mac, entered_mac):
        messagebox.showinfo("Verification", "✅ MAC Verified Successfully!")
```

else:

messagebox.showerror("Verification", "❌ MAC Verification Failed!")

root = tk.Tk()

root.title("Message Authentication Code (MAC)")

root.geometry("600x500")

root.resizable(False, False)

title = tk.Label(root, text="MAC Generator & Verifier (HMAC-SHA256)",
font=("Arial", 16, "bold"))

title.pack(pady=10)

tk.Label(root, text="Secret Key:", font=("Arial", 11)).pack(anchor="w", padx=20)

key_entry = tk.Entry(root, width=60)

key_entry.pack(padx=20, pady=5)

tk.Label(root, text="Message:", font=("Arial", 11)).pack(anchor="w", padx=20)

message_text = tk.Text(root, height=6, width=60)

message_text.pack(padx=20, pady=5)

tk.Button(root, text="Generate MAC", font=("Arial", 11),
bg="green", fg="white", command=generate).pack(pady=10)

tk.Label(root, text="Generated MAC:", font=("Arial", 11)).pack(anchor="w", padx=20)

mac_entry = tk.Entry(root, width=75, state="readonly")

mac_entry.pack(padx=20, pady=5)

tk.Label(root, text="Enter MAC to Verify:", font=("Arial", 11)).pack(anchor="w", padx=20)

verify_entry = tk.Entry(root, width=75)

verify_entry.pack(padx=20, pady=5)

tk.Button(root, text="Verify MAC", font=("Arial", 11),
bg="blue", fg="white", command=verify).pack(pady=15)

root.mainloop()

OUTPUT:-

